Desafio Lighthouse - LH Nautical

Documentacao Tecnica das Respostas

Respostas Finais - Desafio Lighthouse

Este documento foi organizado para responder diretamente ao `docs/desafio.docx`, com foco em clareza, rastreabilidade e aderencia ao que foi pedido.

Status de aderencia ao enunciado (Q1-Q8): **COVERED**.

Questao 1 - EDA

1.1 SQL

```
SELECT
  COUNT(*) AS total_linhas,
  6 AS total_colunas,
  MIN(sale_date) AS data_minima_raw,
  MAX(sale_date) AS data_maxima_raw,
  MIN(total) AS total_minimo,
  MAX(total) AS total_maximo,
  AVG(total) AS total_medio
FROM vendas_2023_2024;
```

1.2 Validacao

- Valor maximo registrado em `total`: **2222973.0**

1.3 Interpretacao

- O dataset tem boa consistencia estrutural (sem nulos e sem duplicatas exatas).
- Existem valores altos em `total`, indicando possiveis outliers.
- Para analise exploratoria inicial, o dataset esta adequado.
- Para modelagem mais sensivel, vale tratar outliers em etapa futura.

Evidencias de execucao (sem tratamento): `notebooks/q1_exploracao_vendas.py` e `data/processed/q1_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\notebooks\q1_exploracao_vendas.py

```
import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "vendas_2023_2024.csv"
OUT_JSON = ROOT / "data" / "processed" / "q1_summary.json"


def main() -> None:
    # Premissa da Q1: apenas leitura bruta, sem limpeza/tratamento.
    df = pd.read_csv(RAW)

    summary = {
        "rows": int(df.shape[0]),
        "cols": int(df.shape[1]),
        "sale_date_min_raw": str(df["sale_date"].min()),
        "sale_date_max_raw": str(df["sale_date"].max()),
```

```
        "total_min": float(df["total"].min()),
        "total_max": float(df["total"].max()),
        "total_mean": float(df["total"].mean()),
        "nulls_by_column": df.isna().sum().astype(int).to_dict(),
        "duplicate_rows": int(df.duplicated().sum()),
    }

    OUT_JSON.parent.mkdir(parents=True, exist_ok=True)
    OUT_JSON.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8")

    print("=== Q1 | EDA vendas_2023_2024 (bruto) ===")
    for key, value in summary.items():
        print(f"{key}: {value}")
    print(f"\nResumo salvo em: {OUT_JSON}")


if __name__ == "__main__":
    main()
```

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q1_summary.json

```
{
  "rows": 9895,
  "cols": 6,
  "sale_date_min_raw": "01-01-2023",
  "sale_date_max_raw": "31-12-2024",
  "total_min": 294.5,
  "total_max": 2222973.0,
  "total_mean": 263797.82826680137,
  "nulls_by_column": {
    "id": 0,
    "id_client": 0,
    "id_product": 0,
    "qtd": 0,
    "total": 0,
    "sale_date": 0
  },
  "duplicate_rows": 0
}
```

Saida observada:

```
=== Q1 | EDA vendas_2023_2024 (bruto) ===
rows: 9895
cols: 6
sale_date_min_raw: 01-01-2023
sale_date_max_raw: 31-12-2024
total_min: 294.5
total_max: 2222973.0
total_mean: 263797.82826680137
nulls_by_column: {'id': 0, 'id_client': 0, 'id_product': 0, 'qtd': 0, 'total': 0, 'sale_date': 0}
duplicate_rows: 0
```

Questao 2 - Produtos

2.1 Codigo Python

Arquivo enviado: `notebooks/q2_normalizacao_produtos.py`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\notebooks\q2_normalizacao_produtos.py

```
# ================================================
# LH Nautical Challenge - Questao 2
# Normalizacao do arquivo produtos_raw.csv
# ================================================

import json
```

```python
from pathlib import Path

import pandas as pd
import unidecode
from rapidfuzz import process

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "produtos_raw.csv"
OUT_CSV = ROOT / "data" / "processed" / "q2_produtos_normalizados.csv"
OUT_JSON = ROOT / "data" / "processed" / "q2_summary.json"

CANONICAS = ["eletronicos", "propulsao", "ancoragem"]


def normalizar_categoria(texto: str) -> str:
    texto_limpo = unidecode.unidecode(str(texto).strip().lower()).replace(" ", "")
    match, score, _ = process.extractOne(texto_limpo, CANONICAS)
    return match if score >= 70 else "outros"


def main() -> None:
    df = pd.read_csv(RAW)

    duplicatas_antes = int(df.duplicated(subset="code").sum())

    df["category"] = df["actual_category"].apply(normalizar_categoria)
    df["price"] = (
        df["price"]
        .astype(str)
        .str.replace("R$", "", regex=False)
        .str.strip()
        .astype(float)
    )

    df = df.drop_duplicates(subset="code", keep="first").copy()

    summary = {
        "rows_before": int(pd.read_csv(RAW).shape[0]),
        "rows_after": int(df.shape[0]),
        "duplicates_removed": duplicatas_antes,
        "unique_categories_after": sorted(df["category"].dropna().unique().tolist()),
        "price_min": float(df["price"].min()),
        "price_max": float(df["price"].max()),
    }

    OUT_CSV.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUT_CSV, index=False)
    OUT_JSON.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8")

    print("=== Q2 | Produtos normalizados ===")
    for key, value in summary.items():
        print(f"{key}: {value}")
    print(f"\nCSV salvo em: {OUT_CSV}")
    print(f"Resumo salvo em: {OUT_JSON}")


if __name__ == "__main__":
    main()
```

2.2 Validacao

- Produtos duplicados removidos: **7**

Evidencia aberta: `data/processed/q2_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q2_summary.json

```
{
```

```
  "rows_before": 157,
  "rows_after": 150,
  "duplicates_removed": 7,
  "unique_categories_after": [
    "ancoragem",
    "eletronicos",
    "propulsao"
  ],
  "price_min": 309.54,
  "price_max": 148198.23
}
```

Questao 3 - Custos de Importacao

3.1 Codigo Python

Arquivo enviado: `notebooks/q3_normalizacao_custos_importacao.py`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\notebooks\q3_normalizacao_custos_importacao.py

```
# ================================================
# LH Nautical Challenge - Questao 3
# Normalizacao do JSON custos_importacao.json
# ================================================

import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "custos_importacao.json"
OUT_CSV = ROOT / "data" / "processed" / "q3_custos_importacao_normalizado.csv"
OUT_JSON = ROOT / "data" / "processed" / "q3_summary.json"


def main() -> None:
    data = json.loads(RAW.read_text(encoding="utf-8"))

    rows = []
    for item in data:
        for hist in item.get("historic_data", []):
            rows.append(
                {
                    "product_id": item.get("product_id"),
                    "product_name": item.get("product_name"),
                    "category": item.get("category"),
                    "start_date": hist.get("start_date"),
                    "usd_price": hist.get("usd_price"),
                }
            )

    df = pd.DataFrame(rows)
    df["start_date"] = pd.to_datetime(df["start_date"], dayfirst=True, errors="coerce")
    df = df.sort_values(["product_id", "start_date"]).reset_index(drop=True)

    summary = {
        "products_in_json": int(len(data)),
        "rows_after_normalization": int(df.shape[0]),
        "start_date_min": df["start_date"].min().strftime("%Y-%m-%d"),
        "start_date_max": df["start_date"].max().strftime("%Y-%m-%d"),
    }

    OUT_CSV.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUT_CSV, index=False)
```

```
    OUT_JSON.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-8")

    print("=== Q3 | Custos de importacao normalizados ===")
    for key, value in summary.items():
        print(f"{key}: {value}")
    print(f"\nCSV salvo em: {OUT_CSV}")
    print(f"Resumo salvo em: {OUT_JSON}")


if __name__ == "__main__":
    main()
```

3.2 Validacao

- Entradas no CSV apos normalizacao: **1260**

Evidencia aberta: `data/processed/q3_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q3_summary.json

```
{
  "products_in_json": 150,
  "rows_after_normalization": 1260,
  "start_date_min": "2016-01-04",
  "start_date_max": "2025-12-31"
}
```

Questao 4 - Dados Publicos

4.1 SQL

```
WITH vendas AS (
  SELECT
    v.id,
    v.id_product,
    v.qtd,
    v.total AS receita_transacao_brl,
    CAST(v.sale_date AS DATE) AS sale_date
  FROM vendas_2023_2024 v
),
custos_vigentes AS (
  SELECT
    c.product_id AS id_product,
    c.start_date,
    c.usd_price,
    LEAD(c.start_date) OVER (PARTITION BY c.product_id ORDER BY c.start_date) AS prox_start_date
  FROM custos_importacao_normalizado c
),
vendas_com_custo_usd AS (
  SELECT
    v.*,
    c.usd_price
  FROM vendas v
  JOIN custos_vigentes c
    ON c.id_product = v.id_product
   AND v.sale_date >= c.start_date
   AND (v.sale_date < c.prox_start_date OR c.prox_start_date IS NULL)
),
base AS (
  SELECT
    x.id_product,
    x.receita_transacao_brl,
    x.qtd,
    x.usd_price,
    p.cotacao_venda,
    (x.usd_price * p.cotacao_venda * x.qtd) AS custo_total_brl,
```

```
    GREATEST((x.usd_price * p.cotacao_venda * x.qtd) - x.receita_transacao_brl, 0) AS prejuizo_transacao_br
  FROM vendas_com_custo_usd x
  JOIN ptax_diario p
    ON p.sale_date = x.sale_date
)
SELECT
  id_product,
  SUM(receita_transacao_brl) AS receita_total_brl,
  SUM(prejuizo_transacao_brl) AS prejuizo_total_brl,
  CASE
    WHEN SUM(receita_transacao_brl) = 0 THEN NULL
    ELSE SUM(prejuizo_transacao_brl) / SUM(receita_transacao_brl)
  END AS percentual_perda
FROM base
GROUP BY id_product;
```

4.2 Validacao

- `id_product` com maior percentual de perda: **72**

4.3 Explicacao

- Data de cambio utilizada: mesma data da venda (`sale_date`).
- Definicao de prejuizo: `max(custo_total_brl - total_venda_brl, 0)`.
- Suposicao relevante: custo em USD considerado unitario, sem impostos e frete.

Evidencia aberta: `data/processed/q4_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q4_summary.json


```
{
  "top_id_product_percentual_perda": 72,
  "top_percentual_perda": 0.6315004931132593,
  "taxa_cambio_usada": "PTAX cotacaoVenda diaria do Banco Central (preenchimento ffill para dias sem cotaca
  "regra_prejuizo": "max(custo_total_brl - total_venda_brl, 0)"
}
```

Questao 5 - Analise de Clientes

5.1 SQL

```
WITH produtos_limpos AS (
  SELECT
    code,
    CASE
      WHEN LOWER(unaccent(actual_category)) SIMILAR TO '%(eletronic|eletronicos|eletro)%' THEN 'eletronicos
      WHEN LOWER(unaccent(actual_category)) SIMILAR TO '%(propuls|propulsao)%' THEN 'propulsao'
      WHEN LOWER(unaccent(actual_category)) SIMILAR TO '%(ancor|ancoraj|encorag)%' THEN 'ancoragem'
      ELSE 'outros'
    END AS category
  FROM produtos_raw
),
base AS (
  SELECT
    v.id_client,
    v.id,
    v.qtd,
    v.total,
    p.category
  FROM vendas_2023_2024 v
  LEFT JOIN produtos_limpos p
    ON p.code = v.id_product
),
clientes AS (
  SELECT
```

```
    id_client,
    SUM(total) AS faturamento_total,
    COUNT(id) AS frequencia,
    SUM(total) / COUNT(id) AS ticket_medio,
    COUNT(DISTINCT category) AS diversidade_categorias
  FROM base
  GROUP BY id_client
),
top10 AS (
  SELECT id_client, ticket_medio
  FROM clientes
  WHERE diversidade_categorias >= 3
  ORDER BY ticket_medio DESC, id_client ASC
  LIMIT 10
)
SELECT
  b.category,
  SUM(b.qtd) AS qtd_total
FROM base b
JOIN top10 t
  ON t.id_client = b.id_client
GROUP BY b.category
ORDER BY qtd_total DESC, b.category ASC;
```

5.2 Validacao

- Categoria mais vendida para os Top 10 clientes fieis: **propulsao**

5.3 Explicacao

- Limpeza de categoria: normalizacao textual + consolidacao de grafias proximas.
- Filtro de diversidade: apenas clientes com 3+ categorias distintas.
- Garantia de escopo: contagem de itens feita somente para os clientes selecionados no Top 10.

Evidencia aberta: `data/processed/q5_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q5_summary.json

```
{
  "top_categoria_qtd": "propulsao",
  "top_categoria_qtd_total": 6030.0,
  "top10_primeiro_id_client": 47,
  "top10_primeiro_ticket_medio": 336859.7039473684
}
```

Questao 6 - Dimensao de calendario

6.1 SQL

```
WITH calendario AS (
  SELECT d::date AS sale_date
  FROM generate_series(
    (SELECT MIN(CAST(sale_date AS date)) FROM vendas_2023_2024),
    (SELECT MAX(CAST(sale_date AS date)) FROM vendas_2023_2024),
    interval '1 day'
  ) d
),
vendas_dia AS (
  SELECT
    CAST(sale_date AS date) AS sale_date,
    SUM(total) AS valor_venda
  FROM vendas_2023_2024
  GROUP BY CAST(sale_date AS date)
),
base AS (
```

```
  SELECT
    c.sale_date,
    COALESCE(v.valor_venda, 0) AS valor_venda
  FROM calendario c
  LEFT JOIN vendas_dia v
    ON v.sale_date = c.sale_date
)
SELECT
  CASE EXTRACT(DOW FROM sale_date)
    WHEN 0 THEN 'Domingo'
    WHEN 1 THEN 'Segunda-feira'
    WHEN 2 THEN 'Terca-feira'
    WHEN 3 THEN 'Quarta-feira'
    WHEN 4 THEN 'Quinta-feira'
    WHEN 5 THEN 'Sexta-feira'
    WHEN 6 THEN 'Sabado'
  END AS dia_semana,
  ROUND(AVG(valor_venda)::numeric, 2) AS media_vendas
FROM base
GROUP BY 1
ORDER BY media_vendas ASC;
```

6.2 Validacao

- Dia da semana com menor media historica: **Domingo**
- Media (2 casas): **3319503.57**

6.3 Explicacao

- O calendario e necessario para incluir dias sem venda no calculo.
- Sem calendario, esses dias ficam ausentes e a media fica artificialmente alta.

Evidencia aberta: `data/processed/q6_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q6_summary.json

```
{
  "pior_dia_semana": "Domingo",
  "pior_media_vendas": 3319503.57
}
```

Questao 7 - Previsao de demanda

7.1 Codigo Python

Arquivo enviado: `notebooks/q7_previsao_demanda_baseline.py`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\notebooks\q7_previsao_demanda_baseline.py

```
# ================================================
# LH Nautical Challenge - Questao 7
# Baseline de previsao com media movel de 7 dias
# ================================================

import json
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = ROOT / "data" / "raw"
OUT_DIR = ROOT / "data" / "processed"
TARGET_NAME = "Motor de Popa Yamaha Evo Dash 155HP"
def parse_sale_date(series: pd.Series) -> pd.Series:
    parsed_iso = pd.to_datetime(series, errors="coerce", format="%Y-%m-%d")
```

```
    parsed_br = pd.to_datetime(series, errors="coerce", format="%d-%m-%Y")
    return parsed_iso.fillna(parsed_br)


def mae(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    return float(np.mean(np.abs(y_true - y_pred)))


def main() -> None:
    vendas = pd.read_csv(RAW_DIR / "vendas_2023_2024.csv")
    produtos = pd.read_csv(RAW_DIR / "produtos_raw.csv")

    vendas["sale_date"] = parse_sale_date(vendas["sale_date"])

    alvo = produtos.loc[produtos["name"] == TARGET_NAME, "code"]
    if alvo.empty:
        alvo = produtos.loc[produtos["name"].str.contains("Motor de Popa Yamaha Evo Dash 155HP", na=False),
    if alvo.empty:
        raise RuntimeError("Produto alvo da questao 7 nao encontrado no catalogo.")
    id_alvo = int(alvo.iloc[0])

    serie = vendas[vendas["id_product"] == id_alvo].groupby("sale_date", as_index=False).agg(qtd_dia=("qtd"

    calendario = pd.DataFrame(
        {"sale_date": pd.date_range(vendas["sale_date"].min(), vendas["sale_date"].max(), freq="D")}
    )
    serie = calendario.merge(serie, on="sale_date", how="left")
    serie["qtd_dia"] = serie["qtd_dia"].fillna(0.0)

    train = serie[serie["sale_date"] <= pd.Timestamp("2023-12-31")].copy()
    test = serie[
        (serie["sale_date"] >= pd.Timestamp("2024-01-01"))
        & (serie["sale_date"] <= pd.Timestamp("2024-01-31"))
    ].copy()

    history = train["qtd_dia"].tolist()
    previsoes = []
    for _, row in test.iterrows():
        pred = float(np.mean(history[-7:])) if len(history) >= 7 else float(np.mean(history))
        previsoes.append(pred)
        # Atualizacao recursiva com valor real do dia para previsao 1-step ahead.
        history.append(float(row["qtd_dia"]))

    test = test.copy()
    test["forecast_qtd_dia"] = previsoes

    metric_mae = mae(test["qtd_dia"].to_numpy(), test["forecast_qtd_dia"].to_numpy())
    soma_semana1 = int(round(test.head(7)["forecast_qtd_dia"].sum()))

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    test.to_csv(OUT_DIR / "q7_previsao_jan_2024.csv", index=False)

    resumo = {
        "id_product_alvo": id_alvo,
        "produto_alvo": TARGET_NAME,
        "mae_jan_2024": metric_mae,
        "soma_previsao_primeira_semana_jan_2024": soma_semana1,
    }
    (OUT_DIR / "q7_summary.json").write_text(
        json.dumps(resumo, indent=2, ensure_ascii=False), encoding="utf-8"
    )

    print("=== Q7 | Previsao baseline ===")
    for k, v in resumo.items():
        print(f"{k}: {v}")
    print(f"\nArquivos salvos em: {OUT_DIR}")
```

```
if __name__ == "__main__":
    main()
```

7.2 Validacao

- Soma da previsao (01/01 a 07/01): **0**

7.3 Explicacao

- Baseline: media movel de 7 dias anteriores.
- Evitei data leakage usando apenas historico disponivel ate o dia anterior.
- MAE em janeiro/2024: **0.9953917050691242**.
- Motivo da soma da primeira semana ser 0: no inicio de janeiro/2024 a janela movel de 7 dias ficou ancorada em historico recente com `qtd_dia` zerada para o produto alvo, resultando em previsao inicial nula.
- Adequacao do baseline: **aceitavel como referencia inicial**, mas limitado para decisao de compra fina sem features adicionais.
- Limitacao: o baseline nao captura sazonalidade longa e eventos externos.

Evidencia aberta: `data/processed/q7_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q7_summary.json

```
{
  "id_product_alvo": 54,
  "produto_alvo": "Motor de Popa Yamaha Evo Dash 155HP",
  "mae_jan_2024": 0.9953917050691242,
  "soma_previsao_primeira_semana_jan_2024": 0
}
```

Questao 8 - Sistema de recomendacao

8.1 Codigo Python

Arquivo enviado: `notebooks/q8_sistema_recomendacao.py`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\notebooks\q8_sistema_recomendacao.py

```
# ================================================
# LH Nautical Challenge - Questao 8
# Recomendacao por similaridade de cosseno (produto x produto)
# ================================================

import json
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
RAW_DIR = ROOT / "data" / "raw"
OUT_DIR = ROOT / "data" / "processed"
TARGET_NAME = "GPS Garmin Vortex Mare Drift"


def cosine_similarity_matrix(matrix: np.ndarray) -> np.ndarray:
    norms = np.linalg.norm(matrix, axis=1, keepdims=True)
    norms[norms == 0] = 1.0
    normalized = matrix / norms
    return normalized @ normalized.T


def main() -> None:
    vendas = pd.read_csv(RAW_DIR / "vendas_2023_2024.csv")
```

```
    produtos = pd.read_csv(RAW_DIR / "produtos_raw.csv")

    inter = vendas.groupby(["id_client", "id_product"]).size().reset_index(name="cnt")
    inter["value"] = 1

    matriz = inter.pivot_table(index="id_client", columns="id_product", values="value", fill_value=0)

    alvo = produtos.loc[produtos["name"] == TARGET_NAME, "code"]
    if alvo.empty:
        alvo = produtos.loc[produtos["name"].str.contains("GPS Garmin Vortex", na=False), "code"]
    if alvo.empty:
        raise RuntimeError("Produto alvo da questao 8 nao encontrado no catalogo.")
    id_alvo = int(alvo.iloc[0])

    product_ids = matriz.columns.to_numpy()
    produto_cliente = matriz.to_numpy().T
    sim = cosine_similarity_matrix(produto_cliente)

    sim_df = pd.DataFrame(sim, index=product_ids, columns=product_ids)
    ranking = sim_df[id_alvo].drop(index=id_alvo).sort_values(ascending=False).head(5).reset_index()
    ranking.columns = ["id_product", "similaridade"]

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    ranking.to_csv(OUT_DIR / "q8_top5_similares.csv", index=False)

    resumo = {
        "id_product_referencia": id_alvo,
        "produto_referencia": TARGET_NAME,
        "id_product_mais_similar": int(ranking.iloc[0]["id_product"]),
        "similaridade_mais_alta": float(ranking.iloc[0]["similaridade"]),
    }
    (OUT_DIR / "q8_summary.json").write_text(
        json.dumps(resumo, indent=2, ensure_ascii=False), encoding="utf-8"
    )

    print("=== Q8 | Sistema de recomendacao ===")
    for k, v in resumo.items():
        print(f"{k}: {v}")
    print(f"\nArquivos salvos em: {OUT_DIR}")


if __name__ == "__main__":
    main()
```

8.2 Validacao

- `id_product` com maior similaridade ao GPS de referencia: **94**

8.3 Explicacao

- Matriz: cliente x produto com presenca (1/0).
- Cosseno mede proximidade de padrao de compra entre produtos.
- Limitacao: metodo ignora ordem temporal e quantidade comprada.

Evidencia aberta: `data/processed/q8_summary.json`

Previa: C:\Users\CodeZ\Desktop\desafio_LH\data\processed\q8_summary.json


```
{
  "id_product_referencia": 27,
  "produto_referencia": "GPS Garmin Vortex Mare Drift",
  "id_product_mais_similar": 94,
  "similaridade_mais_alta": 0.869626356546304
}
```

Material complementar (apoio visual)

- Grafico de prejuizo por produto: `data/processed/q4_prejuizo_por_produto.png`
- Grafico de media por dia da semana: `data/processed/grafico_dias_semana.png`
- Grafico de top clientes por ticket: `data/processed/grafico_clientes_top10_ticket.png`

Checklist de Coerencia (blindagem)

Este checklist foi validado com a execucao real dos scripts no ambiente atual.

- Q1 (bruto): `rows=9895`, `cols=6`, `sale_date_min_raw=01-01-2023`, `sale_date_max_raw=31-12-2024`, `total_max=2222973.0`
- Q2: `duplicates_removed=7`
- Q3: `rows_after_normalization=1260`
- Q4: `top_id_product_percentual_perda=72`
- Q5: `top_categoria_qtd=propulsao`
- Q6: `pior_dia_semana=Domingo`, `pior_media_vendas=3319503.57`
- Q7: `mae_jan_2024=0.9953917050691242`, `soma_previsao_primeira_semana_jan_2024=0`
- Q8: `id_product_mais_similar=94`

Riscos residuais nao bloqueantes:

- Dependencia externa de PTAX quando o cache local nao cobre o periodo.
- Necessidade de reforcar no pitch oral a justificativa da Q7 (`soma_previsao_primeira_semana_jan_2024=0`).

Comando de auditoria rapida (opcional):

```
python -u notebooks/q1_exploracao_vendas.py
python -u notebooks/q2_normalizacao_produtos.py
python -u notebooks/q3_normalizacao_custos_importacao.py
python -u notebooks/q4_prejuizo_por_produto.py
python -u notebooks/q5_clientes_fieis.py
python -u notebooks/q6_dimensao_calendario.py
python -u notebooks/q7_previsao_demanda_baseline.py
python -u notebooks/q8_sistema_recomendacao.py
```

Fechamento

As respostas foram organizadas para ficarem objetivas, rastreaveis e alinhadas ao que foi solicitado no desafio.

If you are looking for a junior who is organized, coachable, and genuinely committed to learning fast and delivering real business value, I would be excited to join your team.

ANEXO - GRAFICOS ADICIONAIS

Graficos gerados automaticamente pelo pipeline a partir dos dados processados.

grafico_clientes_top10_ticket.png

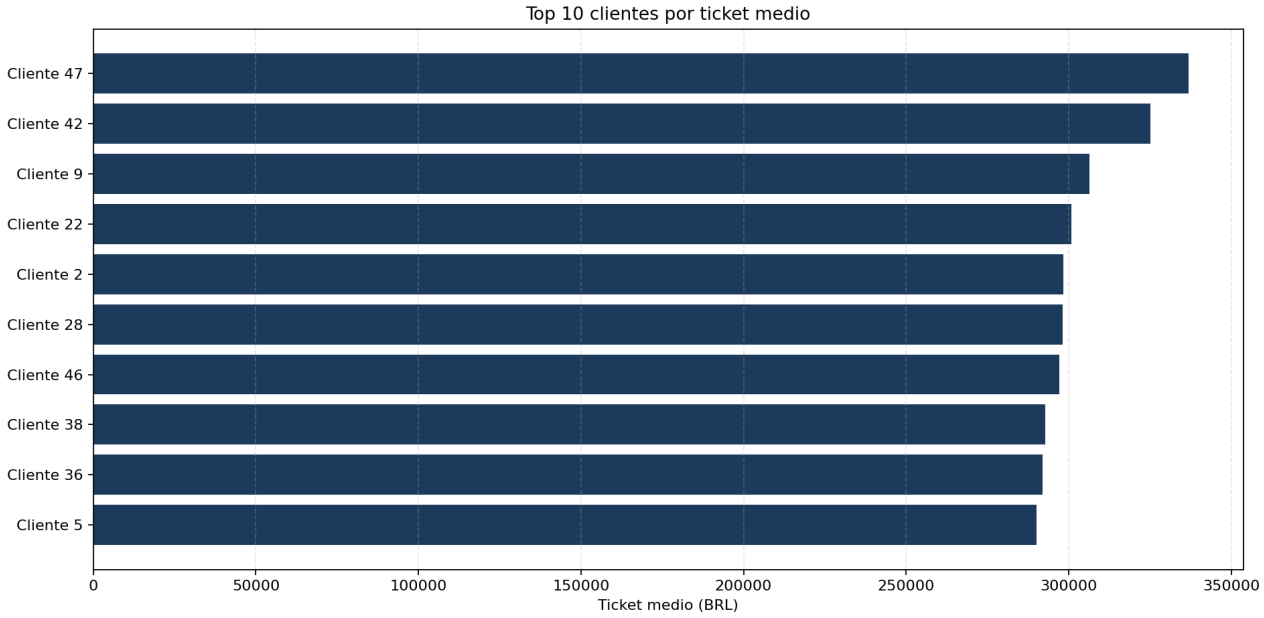


grafico clientes top10 ticket

grafico_dias_semana.png

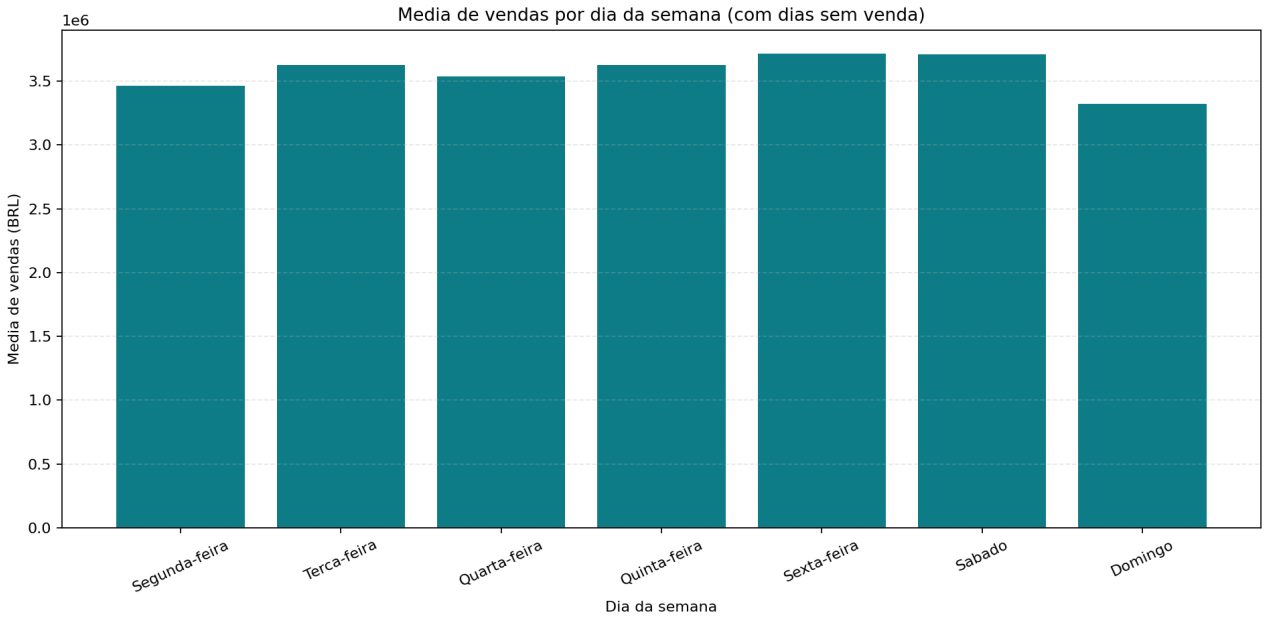


grafico dias semana

grafico_prejuizo_produtos.png

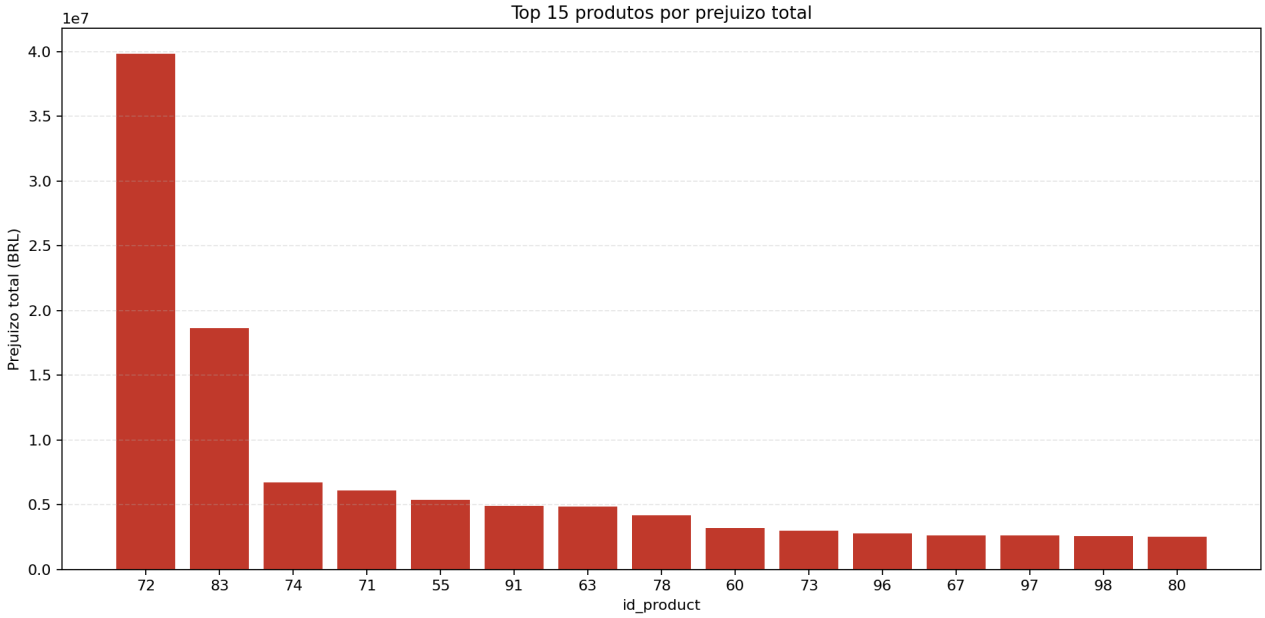
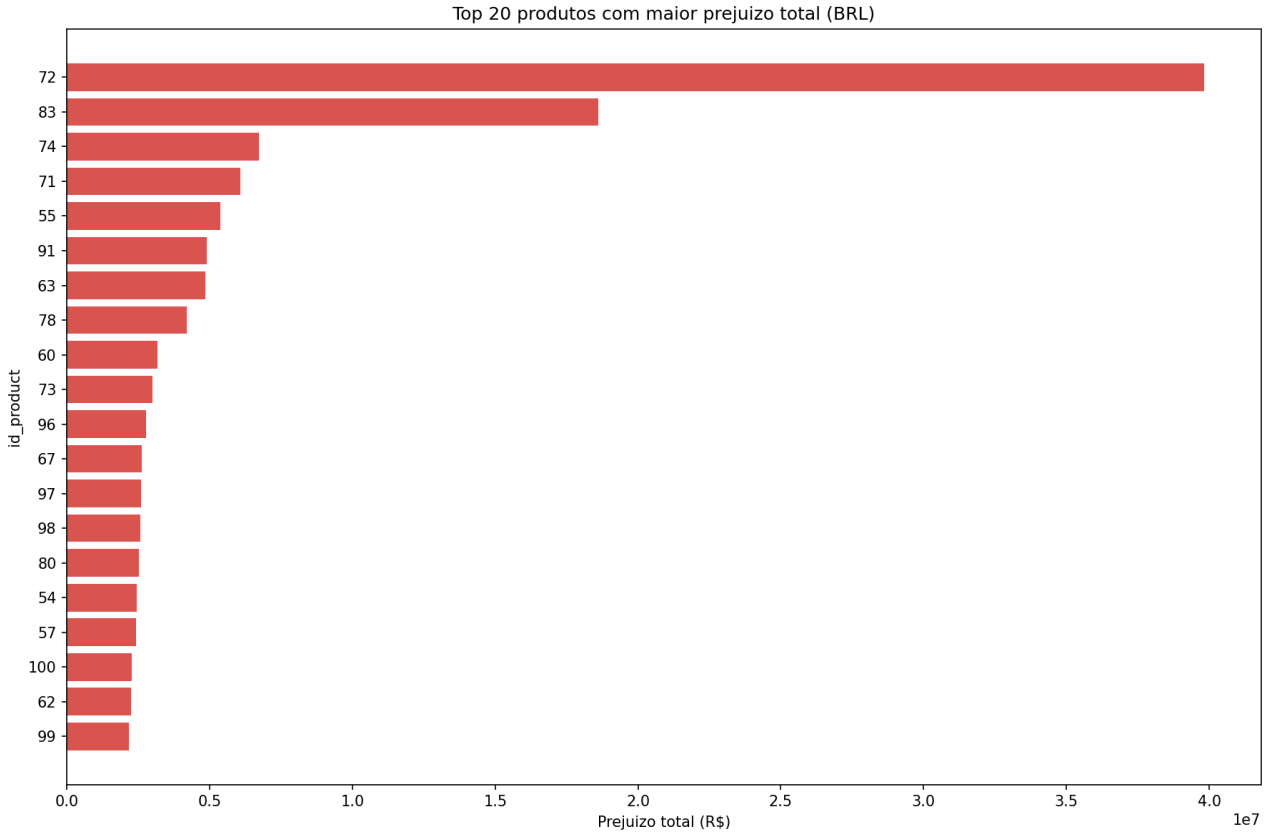


grafico prejuizo produtos

q4_prejuizo_por_produto.png



q4 prejuizo por produto

ANEXO - AUDITORIA AUTOMATICA

Checklist gerado automaticamente na etapa de build do PDF.

Entrada: `C:\Users\CodeZ\Desktop\desafio_LH\arquivos\docs\desafio_resposta_banca_final.md`
Saida: `C:\Users\CodeZ\Desktop\desafio_LH\docs\desafio_resposta_banca_final.pdf`
Gerado em: 2026-03-22 21:20:59 UTC

Verificacao	Status	Detalhe
Cobertura Q1-Q8 no markdown	OK	completo
Artefatos q*_summary.json	OK	todos presentes

Variaveis-chave derivadas dos JSONs processados:

Questao	Variavel	Valor
Q1	total_max	2222973.0
Q2	duplicates_removed	7
Q3	rows_after_normalization	1260
Q4	top_id_product_percentual_perda	72
Q5	top_categoria_qtd	propulsao
Q6	pior_dia_semana	Domingo
Q7	mae_jan_2024	0.9953917050691242
Q8	id_product_mais_similar	94

Fim do documento